Nonlinear Dynamics and Chaos in Orbital Mechanics and Attractor Systems

Prakrit Gajurel

December 16^{th}, 2025

Contact: pgajurel111@gmail.com

1. Introduction

1 Background

Nonlinear dynamics is a branch of mathematics and physics that deals with systems described by equations that are not linear. In such nonlinear systems, small changes in initial conditions can lead to significantly different outcomes, which is the entire concept of chaos. This concept has many implications in many fields including orbital mechanics. Orbital mechanics deals with the motion of objects in space under the influence of gravitational forces. The study of chaos can help us observe how orbits can behave unpredictably due to complex interactions and external forces such as gravitational forces between two or more bodies. A simpler, more pure description of chaos can be seen in attractors, that are states in which a system evolves over time. These attractors have non-periodic and non-repeating behavior that is very sensitive to initial conditions.

2 Purpose

The main purpose of this project is to analyze the role of nonlinear dynamics and chaos theory in the behavior of orbital systems and attractor systems. Nonlinear dynamics as a whole can help explain complex systems where outputs aren't proportional to inputs, and can help in dealing with chaos and turbulence. This also leads to phenomena and thought experiments such as the Butterfly Effect.

3 Scope

This project is limited to only a few attractors and chaotic systems, while there are many more, governed by other mathematical equations. The systems that this project covers are already well-researched on by mathematicians and scientists, and the visualization of the 3 dimensional systems will be discrete, although the clear shape can be seen well. The accuracy or speed of the rendering will depend on the software or the computer limitations.

2. Objectives

- To demonstrate how small changes in initial conditions can cause a noticable difference in the final product of some systems.
- Clear up the concept of chaos theory and nonlinear dynamics.
- Explain the use of the theory in the real world.

3. Proposed Approach

1 Overview

This project will use Python and it's in-built libraries to model some systems governed by mathematical equations. A few renders shall be run, with some slight initial change, and data shall be recorded. The observations will show that a slight change in the initial conditions lead to a noticable, or even a large difference in the resulting outcomes. We will use Matplotlib and some other math libraries from Python to render the animations.

2 Phases

- **Research and Planning**: Look over the theory of chaos, and nonlinear dynamics. Find suitable examples of such systems, and research on their respective theories. Plan software to be used for visualization.
- **Design and Development**: Write Python code to solve and represent the mathematical equations governing the systems, implement a way to store relevant data, write necessary code to visualize the outputs.
- **Testing and Optimization**: Compare results to well-known results obtained by researchers, optimize code for efficiency if needed.
- **Deployment and Maintenance**: Finalize the code, and prepare for the presentation, including the theory/math behind the project and examples.

3 Key Technologies

- Python
- Matplotlib
- NumPy
- SciPy

4. SWOT Analysis

1 Strengths

- Visual approach makes it easy to understand
- Uses open-source tools

2 Weaknesses

- Math such as ODEs and more theory-heavy concepts may be oversimplified
- Limited to a few systems

3 Opportunities

- Can help build a base for understanding more complex, chaotic systems
- Can help in developing advanced computing

4 Threats

- Computational errors
- Overly complex mathematics may cause problems in usability at this academic level

5. Expected Outcomes

This project is expected to produce a fully functional Python-based program that allows for proper visualization of chaotis systems, and how even chaotic systems can have order in them. It is expected that patterns are to emerge in the visualizations of the systems. The observers will be able to observe the behavior in the systems, and notice how initial conditions can heavily affect the final condition.

6. Risk Analysis and Mitigation

- Difficulty solving ODEs - Use SciPy's differential equation solver
- Incorrect visual interpretation - Validate results with known solutions
- Complicated visuals - Keep plots simple and well-labeled

7. Budget Estimate

Omitted as the entirety of the project is based on open-source software.

8. Team Structure

Prakrit Gajurel - Only member in team

9. Conclusion

This project aims to prove that even in chaos, order can be found. The uses of chaos theory and such mathematical concepts are of immense use in the real world, such as in weather forecasting and predicting the motion of heavenly bodies, granted this project is obviously a lot simpler than the models of those. The use of Python for easy computation and visualization allows for understanding of the fact that mathematics can be combined with other sciences to create meaningful results.

Appendix A: Chaos Theory

Chaos Theory is the study of complex, deterministic systems that are very sensitive to initial conditions. That is, even with tiny changes in their initial conditions, it leads to vastly different, unpredictable long-term outcomes, like weather or heartbeats, which can reveal hidden patterns in apparent disorder. The system follows strict rules, but it's behavior appears random and unpredictable due to it being extremely sensitive. One of the most famous metaphors for this branch of mathematics is the Butterfly Effect, where the metaphor says a butterfly flapping it's wings in Brazil can theoretically cause a tornado in Texas. These patterns found in chaotic systems exhibit self-similarity at different scales. Examples of such concepts can be used in weather forecasting, where long-term forecasts become unreliable, and fluid dynamics where turbulence in water and air is common.

Appendix B: Attractors in Chaos Theory and Dynamic Systems

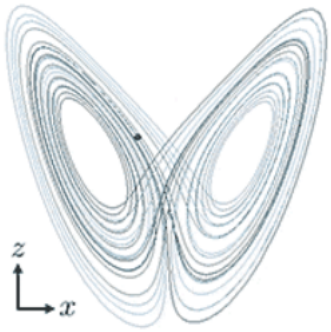
Attractors, in chaos theory, are states or patterns that a dynamic system settles into, acting like a "magnet" for trajectories in space. Chaotic attractors, such as strange attractors, exhibit seemingly random, and unpredictable behaviors, describing the bounded but infinitely complex fractal-resembling shapes. One of the best and most famous examples of a strange attractor is the Lorenz Attractor, found by Edward Lorenz when attempting to create a simple weather model in the early 1960s. It is an attractor with an almost butterfly-shape, governed by three differential equations that result in extremely different results when changed. The differential equations governing the Lorenz Attractor are given below:

$$\frac{dx}{dt} = \sigma(y - x)$$

$$\frac{dy}{dt} = x(\rho - z) - y$$

$$\frac{dz}{dt} = xy - \beta z$$

Here, σ is known as the Prandtl number, ρ is the Rayleigh number, and β relates to the physical dimensions of the fluid layer.



A sample solution in the Lorenz attractor when $\rho = 28, \sigma = 10, \beta = \frac{8}{3}$.

The data can be plotted in Python by assigning values to dt as a small change in time, usually 0.01, then solving for dx, dy, dz, and incrementing the respective variables by their small change, and plotting them using a software like Matplotlib.

Appendix C: Chaos in Pendulums

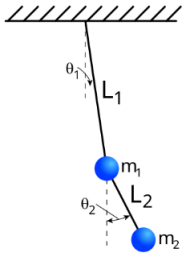
Chaos in pendulums arise in double pendulums or higher. As usual with chaos, tiny changes in the initial condition(which could be the ratio of the lengths of the sides between the bobs of the pendulum) result in a completely different outcome. A double pendulum consists of two pendulums attached end to end. This project only deals with double pendulums, which are extremely difficult to work with, so the concepts have been simplified heavily(solving the equations of the center of masses requires math such as the Lagrangian).

If, in the Cartesian plane, the origin is taken to be the point of suspension of the double pendulum, define θ_1 and θ_2 to be the angles between each limb and the vertical(or the y-axis in this case), then we have the following for the coordinates of the center of mass of the first pendulum:

$$x_1 = \frac{1}{2}\ell \sin(\theta_1)$$
$$y_1 = -\frac{1}{2}\ell \cos(\theta_2)$$

Similarly, the coordinates for the center of mass of the second pendulum is given to be:

$$x_2 = \ell(\sin(\theta_1) + \frac{1}{2}\sin(\theta_2))$$
$$y_2 = -\ell(\cos(\theta_1) + \frac{1}{2}\cos(\theta_2))$$



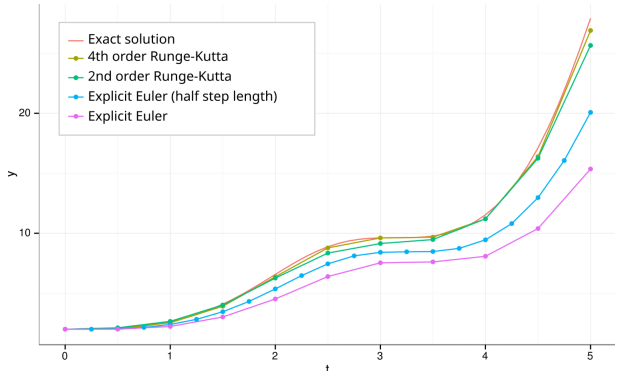
A double pendulum, with the respective variables assigned.

These coordinates act as the parameters for the differential equations we must solve to visualize the pendulum. Simulating a double pendulum in Python now requires numerically solving the differential equations, and animating the results using libraries like Matplotlib.

Appendix D: Numerically Solving Ordinary Differential Equations

Solving these equations numerically in Python can be used by numerical integration methods, which work by taking tiny time steps to estimate the pendulum's position and velocity over time. We could introduce new variables for the angular velocities and get a system of first-order ODEs.

Solving the ODEs can be used by explicitly doing Euler's Method or Runge-Kutta, but the python library SciPy already has those methods in-built into it's ODE solvers.



A comparison of the Runge-Kutta and Euler methods for numerically solving the differential equation $y' = \sin(t^2) \cdot y$. The 4th order Runge-Kutta(RK4) (in yellow) is the closest approximation to the actual solution(in red).